Design notes — why this is built the way it is

This is the reasoning document. README.md says how to run the tool; this says why each decision went the way it did, what I rejected, what I changed mid-build and why, and what I would do differently with more time.

---

1. The framing

The brief asks two questions per repo — *is it secure* and *does everyone with access still deserve it* — and one meta-question: can a non-engineer act on the answer in two minutes.

The second question is the hard one, because it is a **judgment** dressed as a data problem. "Does this person deserve access" has no ground truth in the API. Whatever number I produce is an argument, so the design goal was not accuracy — it was **defensibility**: every number must be explainable in one sentence to the person whose access is being questioned.

That single goal drives most of what follows: the log damping, the review weighting, the smooth decay, the exclusion panel, and the decision never to guess at an unattributed commit.

---

2. Contribution scoring

```
activity = 3.0·ln(1+commits) + 2.5·ln(1+reviews)
         + 2.0·ln(1+PRs merged) + 1.0·ln(1+PRs opened)

score    = activity × 0.5^(days since last activity / 180)

risk     = permission weight / (1 + score)
```

Why log damping

Contribution is not linear. The difference between 0 and 5 commits is the interesting one; the difference between 200 and 205 is noise. `log1p` encodes that, handles zero without a special case, and stops one prolific month from making someone look permanently active. The test suite pins this: the 0→5 gain must exceed the 200→205 gain by more than 20×.

Why reviews are weighted at 2.5, nearly as high as commits

This is the most consequential number in the model. A naive commit-count review flags exactly the wrong person: the staff engineer or security reviewer who writes little code but reviews constantly, and whose `maintain` access is the most load-bearing on the team.

The fixture encodes this case deliberately — `priya-staff` has 2 commits and 21 reviews on `payments-api`, and a dedicated test asserts she is never flagged on any repo. If someone changes the review weight to something small, that test fails and says why.

Why a half-life instead of a cutoff

Access risk is about *now*. Someone with 300 commits who vanished 18 months ago should not outrank someone with 4 commits last week — and a hard "active in the last 180 days" cutoff would make a person at 179 days safe and at 181 days flagged, which is impossible to defend in the conversation that follows.

A 180-day half-life degrades smoothly and matches how people actually reason about staleness. It also makes the boundary uninteresting, which is the point.

Why `risk = weight / (1 + score)`

Score alone answers "who is inactive". A reviewer triaging a list needs "who is inactive **and dangerous**". A dormant admin and a dormant read-only account are both inactive; only one of them can delete the repository. The `1 +` keeps the expression finite at score 0, where risk equals the raw permission weight — a convenient property, since the top of the list reads directly as "this person holds admin and has done nothing."

Threshold: 5.0

Calibration, with fresh activity:

ACTIVITY	SCORE
1 commit	2.08
5 commits	5.37
10 commits	7.19
3 reviews only	3.47
11 reviews only	6.21
Nothing at all	0.00

So 5.0 means roughly *"fewer than four recent commits and no meaningful review activity."* It is a policy number, not a mathematical one, which is why it sits in `config.py` with that table in a comment rather than being buried in the scoring code.

What I rejected

- **GitHub's own `contributors` endpoint.** It counts commits on the default branch for all time, with no window and no review data. It would have been one call per repo instead of a GraphQL query — but it cannot see reviews, which makes it useless for the exact case the model exists to protect.
- **Lines changed / diff size.** Trivially gamed, and punishes people who delete code or work on config-heavy repos.
- **Weighting by recency of each individual event** (rather than decaying the aggregate). More "correct", significantly harder to explain, and it moves almost nobody across the threshold. Not worth the loss of explainability.
- **A single org-wide score per person.** The brief asks for per-repo, and it is the right unit: someone can be the most active person in the org and still hold stale admin on a repo they left two years ago.

---

3. Access: three kinds, never merged

The brief calls out "teams vs. individuals" as a focus area, and it is where a naive implementation quietly loses information.

GitHub grants repository access three ways:

KIND	HOW TO REVOKE IT
Direct	One click on this repo
Outside collaborator	One click, but the person is not an org member
Team-inherited	Change the team — affects every repo that team can reach
Org base permission	One org-wide setting — affects every member on every repo

The fourth one is the trap, and I only found it by running against a real organization. `default_repository_permission` (Settings → Member privileges) can hand every member the same access to every repository. Those people appear in the `all` collaborator listing but not in `direct`, and they belong to no team — so the obvious `all - direct` rule files them as team-inherited and tells the reviewer to go change a team that does not exist. The scanner now reads the org setting, separates that path, and points the remediation at the one place it can actually be fixed.

These are different findings with different remediations, so they are stored in separate columns (`is_direct` / `is_outside` / `is_team`, each with its own permission) and labelled separately in the report. A person holding both direct and team access keeps both, visibly, on one row. `effective_permission` exists as a derived convenience and is explicitly not authoritative.

How team access is detected. GitHub has no `affiliation=team`, so the scanner fetches `direct`, `outside` and `all`, and treats `all - direct` as team-inherited. To *name* the responsible team it indexes the org's teams once per run (1 + T requests for the whole org, not per repo) and intersects with each repo's team list. A suggestion then reads "via Platform Engineering" instead of just "inherited", and carries the right remediation advice.

Stated limitation: a member holding *both* a direct grant and team access is reported as direct. The REST listing reports one effective permission per person and does not decompose it, so claiming to know the team component for that person would be a guess. The scanner says direct rather than inventing detail.

Team-inherited grants **are** included in the suggestion list, labelled, with an action column that says removing them affects every repo that team can reach. Excluding them would hide the place stale access most often accumulates.

---

4. Edge cases

Handled explicitly, each with a test:

CASE	TREATMENT
Empty repos	The commits endpoint returns 409. Raised as its own `EmptyRepositoryError`, tagged, excluded from suggestions — never swallowed as "no data".

CASE	TREATMENT
Archived repos	Scanned and tagged, *not* skipped. Stale admin on an archived repo is still live access, and any holder can unarchive it. In the demo the single highest-risk finding sits on an archived repo. Archiving changes the *remediation advice* and never the score: GitHub refuses collaborator changes while a repo is archived, so the report tells the reviewer to remove the grant at org or team level rather than sending them to a settings page that will reject them.
Forks	Skipped by default (`SKIP_FORKS`), because contribution history on a fork belongs mostly to the upstream project. Configurable, and the skip is recorded as a row so the reader can see it happened.
Unattributed commits	When a commit's author email is not linked to a GitHub account, `author.user` is null. These are counted, totalled and reported — never attributed. Matching on raw name or email would silently credit the wrong person, and a review that quietly misattributes work is worse than one that admits a gap.
Bots	Excluded by `type == "Bot"` or a login ending in `[bot]` . A test asserts a human named `robotnik` is still assessed.
Org owners	Never flagged, checked first so the recorded reason is stable for an owner who is also allowlisted. If the token cannot read the owner list, the scanner logs an error saying suggestions must be reviewed more carefully — it does not silently proceed as if there were no owners.
New repos	Excluded from suggestions, still scanned for advisories.
Single-contributor repos	Excluded — with nobody to compare against, a low score says nothing.
Admins with legitimately low commit activity	This is the review weighting plus the allowlist. An engineering manager who reviews but does not commit scores well; one who does neither is flagged, which is arguably correct — and if it is not, that is what `LOGIN_ALLOWLIST` is for.
Custom repository roles	An unrecognised permission gets the *write* weight (1.5), not the read weight. Guessing "harmless" about an unknown role is the wrong way to be wrong.
Revoked access between runs	Collaborator rows are deleted and re-inserted inside one transaction, so someone whose access was removed does not linger in the report forever.

Deliberately not handled, and why:

- **Nested pagination of PR reviews.** Reviews are read `first: 50` per PR. A PR with more than 50 reviews records a truncation note rather than paginating — it is rare, and it cannot change a flag decision by enough to matter.
- **Commits outside the default branch.** GraphQL history walks the default branch only. Long-lived feature branches are invisible. A full ref walk is dramatically more expensive for a marginal accuracy gain.
- **GitHub Apps / Actions identities that are not typed as `Bot` .** A service account created as a normal user looks human. The allowlist is the intended remedy.
- **Deploy keys, SSH keys and PATs as access paths.** Real stale-access vectors, out of scope for the brief.

---

5. API efficiency and behaviour on a large org

The brief asks how this behaves against 100+ repos. Concretely:

Per repo: 3 collaborator listings (direct / outside / all) + 1 repo-teams listing = ~4 REST requests, plus 2 GraphQL queries (commits, PRs+reviews).

Once per org: repo list (~1–2 pages), owners, members, team index (1 + T), and — critically — **advisories in a single org-wide paginated call rather than one per repo.** For 100 repos that is ~1–3 requests instead of 100.

So a 100-repo scan costs roughly **410 REST requests** against a 5,000/hour quota: about 8% of budget. A 1,000-repo org lands near 4,000 and the client's pre-emptive pausing becomes load-bearing rather than theoretical.

Four mechanisms keep it inside the limits:

1. **Pre-emptive pausing.** `X-RateLimit-Remaining` is read from every response and the client pauses *before* the quota is gone, not after the 403.
2. `Retry-After` **honoured** on both 403 and 429, including the integer and HTTP-date forms, and including GitHub's secondary rate limit.
3. **ETags on every GET.** A 304 does not count against the quota, so a re-run over a mostly-unchanged org is close to free. The cached envelope stores the `Link` header alongside the body, so a 304 on page 3 still knows where page 4 is — without that, conditional requests silently break pagination.
4. **One GraphQL query per repo, not per member.** For a repo with 40 collaborators this is 2 requests instead of 80. This is the single biggest efficiency decision in the project.

A 403 from GitHub is overloaded, so the client classifies it as `primary` / `secondary` / `permission` before acting. Getting this wrong either burns the run retrying a permission error, or hammers the API on a real limit. "Dependabot is disabled for this repo" and "you are out of quota" arrive as the same status code and must not be handled the same way.

---

6. Idempotency and resume

- Every fact table is keyed on natural GitHub identifiers and written with `INSERT ... ON CONFLICT DO UPDATE`. A second run updates rows in place.
- Every fact row carries the `run_id` that last touched it, so a stale row left from an earlier run is visible rather than silently blended in.
- `run_progress` records `(run_id, repo_id, stage)` and is committed per repo, so `--resume` continues at the next unfinished repo instead of refetching.
- A test seeds the fixture org twice and asserts the row counts do not move.

The advisory key is not the obvious one. `ghsa_id` looks like the natural primary key for an advisory, and it is wrong. A single repo can carry several open alerts for the same GHSA in different manifests — two lockfiles pinning the same vulnerable package — so keying on `(repo_id, ghsa_id)` alone silently drops all but one, and silently dropping real vulnerabilities is the worst failure mode this tool has. The key is `(repo_id, ghsa_id, manifest_path)`; `ghsa_id` stays indexed for cross-repo grouping. I found this by reading an actual Dependabot payload rather than by reasoning about the schema, which is the argument for checking a real response before designing a table around it.

---

7. Dashboard decisions

The "readable in two minutes" requirement is a design constraint, not a nicety.

- **Answers before data.** The page opens with a plain-language "short version" — *4 critical or high vulnerabilities are still open; the oldest has been open for 347 days; 10 access grants look stale, 2 of them admin.* The tables below are evidence for those sentences, not a puzzle to solve.
- **Evidence beside every suggestion.** A removal suggestion with no dates next to it is an accusation. Each row carries last commit, last review, permission, how the access was granted, the sentence explaining the flag, and what the reviewer would actually have to do to act on it.
- **The exclusion panel is a feature.** A reviewer who cannot see who was skipped has no reason to trust the list of who was not. Section 4 names every excluded person and repo with the reason.
- **"Nothing has been changed" appears in the section header**, not a footnote.
- **Self-contained output.** Chart.js is vendored and inlined, so the file works from an email attachment or a USB stick with no server and no network.

---

8. What changed during the build, and why

Four things moved after I started, each because building the thing exposed something the plan had not:

1. The advisory primary key. Described above — discovered while writing the upsert, because the multi-manifest case is obvious once you look at a real alert payload.

2. `last_commit_ever` was added. The GraphQL history is window-scoped, so for a person whose last commit predates the window there is no date at all — and that is exactly the person a reviewer needs a date for. The evidence column would have read "never" for the most important rows. Fixed with one cheap REST lookup (`per_page=1`) **for flagged members only**, so the cost is bounded by the size of the suggestion list rather than the size of the org.

3. An ETag caching bug, caught by a test. The cache guard read `if not (self.use_cache and self.cache)` . A cache object that is empty is falsy, so on a fresh database caching was disabled for the entire first run — and, because nothing was ever stored, for every subsequent run too. It now checks `self.cache is None` . There is a regression test named after the mistake. This is the clearest argument in the project for testing the HTTP layer against a mock transport rather than assuming it works.

4. A refused access listing rendered as "nobody has access". Found by pointing the scanner at a real token that lacked repository `Administration: read` . The collaborator listings returned 403, which the code treated as an allowed-empty result, so the dashboard would have stated confidently that no one had access to any repository. For a security report that is the worst possible failure: a wrong answer delivered with confidence. The three listings are now read through a helper that captures the refusal, `AccessSnapshot.complete` distinguishes "empty" from "unreadable", and the dashboard leads with a red banner naming every repository whose access could not be read. `tests/test_access.py` covers it.

5. Org base permission was being reported as team-inherited. Described in section 3. The scan of a live organization with `default_repository_permission = "read"` produced a member labelled "via a team" when the org had no teams at all. Both the label and the remediation advice were wrong, and the error is invisible on any org that leaves base permissions at *none* — which is why fixtures alone never caught it.

6. Contributors without access were being suggested for access removal. Also from the live scan. A repository's contributor list and its collaborator list are different sets: a coding agent's commits were attributed to an account that held no permission on the repo, so it was scored, fell below the threshold, and would have been recommended for the removal of access it never had. `MemberInput.has_access` now gates the suggestion — such people stay in the per-repo table as context, because "someone wrote half this code and has no access today" is useful information, but they can never be suggested for removal. Nonsense recommendations are how a report loses its reader.

7. The report's date filters used the wall clock. Every "N days ago" in the template called `datetime.now()` instead of the run's timestamp, so the demo — whose whole purpose is reproducible output — would have drifted day to day, and the suggestion table could disagree with itself by one day. The filters are now bound to the run's clock at render time.

---

9. What I would change for production

Correctness and coverage - Read commits from all refs, not just the default branch, for repos where feature-branch work dominates. - Track access *changes* between runs — "granted admin 3 days ago, never used it" is a stronger signal than any snapshot, and the schema already carries `run_id` per row, which is most of the work. - Use the audit log API (Enterprise) to distinguish "inactive" from "never had a reason to be active".

Operations - ~~Run it as a scheduled job~~ — **built**, see `.github/workflows/weekly-scan.yml` and section 10. Still to do: publish the dashboard to Pages rather than to a build artifact, so the Slack message can link to a page instead of to a download. - Open a tracking issue per suggestion, assigned to the repo owner, with a 30-day auto-close — turning a report into a workflow. The report is a precondition for that, not a substitute. - Async or process-parallel repo scanning. Repos are independent and the per-repo work is I/O-bound; the sequential loop is the obvious next bottleneck past a few hundred repos. It was not worth the complexity at this size, and it would have made rate-limit handling meaningfully harder to reason about.

Quality - Configurable per-team thresholds. A `security` team's dormancy tolerance differs from a `sandbox` team's. - A feedback loop: record which suggestions a human accepted or rejected, and report the false-positive rate. Without that, the threshold is forever a guess and the model can never improve. - Structured logging with a run correlation ID, and metrics on API spend per run.

---

10. Challenges and notes on tooling

Why code rather than n8n / Make. I evaluated a no-code approach and chose code for three reasons specific to this problem:

1. The scoring logic is the deliverable's core argument. It needs unit tests against hand-computed values, which is awkward-to-impossible to express in a visual workflow.
2. Correct rate-limit handling — pre-emptive pausing, `Retry-After`, secondary limits, ETag revalidation with cached `Link` headers — is not something the HTTP nodes in those platforms expose. Most workflow builders end up retrying blindly, which is exactly the behaviour that gets a token throttled.
3. Idempotent resume needs transactional storage keyed on natural IDs. n8n's state model does not offer that without an external database anyway.

The brief prefers "*building the capability to automate using code and a combination of existing tools, rather than building everything from scratch*" — so I leaned on `httpx`, `Jinja2`, `Chart.js` and SQLite for everything except the judgment, and wrote from scratch only the parts that *are* the judgment: the scoring model, the exclusion rules, and the API-etiquette layer.

Where the workflow platform does belong: orchestration, not logic. Rejecting n8n for the *analysis* is not the same as rejecting scheduled automation, so the tool ships with the orchestration layer it was designed for — `.github/workflows/weekly-scan.yml`. The split is deliberate:

LAYER	OWNER	WHY
Scan, score, rank	`main.py`	Needs unit tests, transactional state and precise rate-limit behaviour
Schedule, notify, publish	the workflow platform	Needs cron, secrets and a Slack integration, none of which belong in application code

`main.py --json` writes a machine-readable summary to stdout for exactly this reason; the workflow reads it, posts the top five suggestions to Slack only when there is something to act on, and uploads the dashboard as a build artifact. The scan database is restored from cache between runs so week two revalidates with ETags instead of refetching the org.

Note what the workflow deliberately does not contain: any step that removes access. The automation is built right up to the point of action and stops there. That boundary is the brief's central constraint, and it is easier to defend as a line in a YAML file that a reviewer can read than as a promise in a document. The same workflow shape would port to n8n almost node-for-node — Schedule Trigger → Execute Command → IF → Slack — which is the honest reason I did not also build it there: it would demonstrate the same judgment twice.

Genuine difficulties encountered

- **GitHub has no `affiliation=team`.** Isolating team-inherited access requires a set difference, and naming the team requires a separate index. Getting this precise took longer than any other part of the scan.
- **A 403 means four different things.** Classifying it correctly is the difference between a scan that recovers and one that either aborts or hammers the API.
- **GraphQL reports failures with HTTP 200.** Status codes are meaningless there; the `errors` array with its `type` field is the real signal, and `RATE_LIMITED`, `NOT_FOUND` and `SERVICE_UNAVAILABLE` each need different handling.
- **Window-scoped queries cannot answer "when did they last commit?"** — see change #2 above. The obvious data model was subtly wrong for the report's most important column.

Features built on top of the base requirement

- `--demo` mode: the full pipeline on fixtures with a pinned clock, so the tool can be demonstrated and verified without a token, an org, or paid security features — and so this submission's numbers can be reproduced exactly.
- `--resume`, backed by per-repo per-stage progress.
- `--report-only`, which re-renders a dashboard from a database captured days earlier with no API calls.
- Named team attribution in the suggestion list, with remediation advice that differs for team-inherited grants.
- All-time last-commit lookup for flagged members, bounded by suggestion count.

- Determinism: stable tie-breaks in the ranking so two runs over unchanged data produce an identical report apart from the run number and generation timestamp, which exist precisely to vary. That makes the output diffable week over week: anything that changes is a real change in the organization.
- A weekly scheduled scan that notifies on findings and revokes nothing, plus CI that runs the suite on two Python versions and then fails the build if a `--demo` run stops producing the findings this documentation claims.
- A print stylesheet on the dashboard, so "Save as PDF" produces a complete report rather than one with every collapsed repository section missing.

What I would build next, given more time: the accept/reject feedback loop. Everything else here is mechanism; that is the only thing that would make the threshold defensible with evidence rather than argument.

---

11. AI assistance disclosure

The brief asks for this explicitly, so here it is plainly.

Tool used: Claude (Anthropic), via Claude Code, as a pair-programming assistant throughout.

What it was used for: scaffolding the module structure; drafting the HTTP client, SQLite schema, GraphQL queries, Jinja template and test suite; and producing this documentation.

Where its output was checked or corrected:

- **The scoring constants were specified by me, not chosen by the model,** and the test suite verifies the implemented arithmetic against hand-computed values (`3.0·ln(1+10) = 7.1936858`) rather than against the model's assertion that it worked.
- **The ETag cache bug** (section 8, item 3) was in AI-drafted code and was caught by running the tests, not by reading them. It would have silently disabled caching on every fresh run.
- **The wall-clock bug in the report filters** (section 8, item 5) was likewise AI-drafted and caught by comparing two numbers on the rendered page that disagreed by one day.
- **Three bugs were found only by pointing the tool at a real organization**, not by any test: the silent-empty-access bug (item 4), the base-permission mislabelling (item 5), and the nonsense suggestions for contributors with no access (item 6). All three are invisible against fixtures, because fixtures encode what I already believed was true. That is the argument for running against real data early, and it is why the run report keeps a live section even though the demo org is more illustrative.
- **The silent-empty-access bug** (section 8, item 4) was only found by running against a real token with incomplete permissions. No unit test would have caught it, because the code did exactly what it was written to do — the mistake was in deciding that a 403 was an acceptable empty result.
- **The advisory primary key** was changed away from the specified `ghsa_id` after checking a real Dependabot payload and finding the multi-manifest case.
- **The expected ranking in `tests/test_pipeline.py` was computed by hand** and the implementation was made to match it, rather than the test being written from whatever the code happened to produce. That ordering — including the three-way tie at risk 1.50 resolved by login — is the strongest single check in the suite.

A second pass, before submission. I also had the finished submission reviewed by the same assistant against the task brief, and acted on what it found. Two things it caught were real defects rather than polish:

- **The remediation advice was wrong for archived repositories.** The report told reviewers a stale grant "can be revoked on this repository alone" — but GitHub refuses collaborator changes while a repo is archived, so that advice cannot be followed. It affected four of the ten suggestions in the demo, including the second-highest-risk finding. Fixed, with three tests.
- **The same advice was written twice**, in `score.py` and again inline in `report.py`, and the two copies had already drifted apart. They are now one function that both call, which is why the archived case only had to be fixed once.

It also caught a documentation claim I could not support — an earlier draft of section 6 said the brief "specified keying advisories on `ghsa_id`" when the brief says nothing about keys — and two transposed test counts in `RUN_REPORT.md`. Both are now corrected. I mention this because a review that only ever confirms your own work is not worth much, and because the class of error is consistent with the rest of this section: the bugs were in paths a happy-path manual run never exercises, and the documentation errors were claims nobody had checked against the source.

The honest summary: the assistant was substantially faster at producing correct-shaped code than I would have been alone, and produced three bugs that looked completely reasonable on the page. Two were caught by executing the tests against a mock transport; the third only surfaced when the tool was pointed at a real token with incomplete permissions. All three were in error-handling paths that never fire in a happy-path manual run. That is the argument for the test suite, and it is why I would not ship AI-drafted infrastructure code without one.
